

CSharp compacted

Grimy86

2026

Contents

1	Entry point	1
2	Data types	3
2.1	Value types	3
2.2	Reference types	3
3	Variables	5
4	Operators	7
4.1	Arithmetic	7
4.2	Assignment	7
4.3	Comparison	7
4.4	Logical	8
5	Control flow	9
5.1	If, if-else, else-if & ternary operator	9
5.2	Switch	10
5.3	While & do-while loop	10
5.4	For & foreach loop	10
5.5	Break & continue	11
6	Type conversion and parsing	13
6.1	Casting compatible types	13
6.2	As conversion	13
6.3	Conversion methods	14
6.4	Parsing strings	14
7	Functions / Methods	15
7.1	Working with methods	15
8	Exception handling	17
8.1	What is an exception?	17
8.2	Throwing exceptions	17
8.3	Handling exceptions	17
8.4	Creating user-defined exceptions	18
9	Preprocessor directives	19

10 Data collections	21
10.1 Summary	21
10.2 Arrays	21
10.3 Enums	22
10.4 Lists	22
10.5 Dictionaries	22
10.6 Queues	22
10.7 Stacks	23
10.8 HashSet	23
10.9 SortedList	23
11 Object-Oriented Programming (OOP)	25
11.1 Why OOP?	26
11.2 The four pillars of OOP	26
11.2.1 1. Encapsulation — keep data safe inside objects	26
11.2.2 2. Abstraction — hide unnecessary details	27
11.2.3 3. Inheritance — reuse and extend existing code	27
11.2.4 4. Polymorphism — one interface, many implementations	27
12 Nullable value types	29
12.1 How it works internally	29
12.2 Null-coalescing operator (??)	29
12.3 Null-conditional operator (?.)	30
12.4 Converting nullable to non-nullable	30
13 Pass by reference	31
13.1 ref	31
13.2 out	31
14 Tuples	33
15 Delegates	35
15.1 Passing delegates	35
15.2 Multicast delegates	36
15.3 Generic delegates	36
15.4 Built-in generic delegates: <code>Func</code> , <code>Action</code> , <code>Predicate</code>	36
15.5 Anonymous methods	36
15.6 Use cases	37
16 Lambda expressions	39
16.1 Two types of lambdas	39
17 Expression-bodied members	41
18 Generics	43
19 Extension methods	45
20 Indexers	47

21 Operator overloading	49
22 Partial	51
22.1 Partial classes	51
22.2 Partial methods	52
23 Method hiding	53
23.1 Intentional hiding with the new keyword	54
24 Events	55
24.1 Observer design pattern	55
24.2 Declaring an event	55
24.2.1 Publisher class	55
24.2.2 Subscriber class	56
24.2.3 Simplified example	56
24.3 Built-in EventHandler delegate	56
24.4 Passing event data	57
25 Attributes	59
25.1 Predefined attributes	59
25.2 User-defined attributes	60
26 Reflection	61
27 Language Integrated Query (LINQ)	63
27.1 LINQ methods	64
28 Multithreading / Asynchronous programming	65
28.1 Threads (System.Threading)	65
28.2 Tasks (System.Threading.Tasks)	66
29 The IDisposable interface	67
29.1 How to implement IDisposable	67
29.2 Real-world example — SqlConnection wrapper	68
30 Unsafe	71
31 Dependency Injection (DI)	73
31.1 Constructor injection	73
31.2 Property injection	74
31.3 Method injection	74
31.4 Using an IoC container	75
32 WPF Dependency Properties (DP)	77
32.1 Creating a dependency property	77
32.2 Attached dependency properties	78
33 ASP.NET Core Web API	79
33.1 REST constraints	79
33.2 HTTP verbs and CRUD mapping	79

33.3 HTTP status codes	80
33.4 Project setup	80

Chapter 1

Entry point

C#'s entry point, like most compiled languages, is the **Main** function. It is the first function executed when the program starts. For now there is no need to worry about **static**, **void**, or **string[]**.

A typical C# program looks like this:

```
// Import the 'System' namespace so we can use its classes and functions.
using System;

// A container that holds and organizes classes.
namespace HelloWorld
{
    // A container for data and methods - a key part of OOP.
    class Program
    {
        // Entry-point: the function the runtime calls first.
        static void Main(string[] args)
        {
            // A function imported from the 'System' namespace.
            Console.WriteLine("Hello World!");
        }
    }
}
```

Key points:

- `using System;` — imports the namespace so its types are available without full qualification.
- `namespace` — a container that organizes classes, prevents naming collisions.
- `class Program` — the class that owns `Main`.
- `static void Main(string[] args)` — the entry-point signature the runtime expects.

Chapter 2

Data types

Variables are containers that store a value belonging to a **data type**. C# has two categories of types: **value** and **reference**.

- Value types store the actual value **directly** in memory.
- Reference types store a **reference** (memory address) that points to the value stored on the heap.

We can use built-in types defined by the language or define our own.

2.1 Value types

Predefined:

Real-world concept	C# keyword	Example
Whole numbers	<code>int</code>	-5 or 5
Floating point numbers	<code>float</code> or <code>double</code>	-5.5 or 5.5
Single letters	<code>char</code>	'A' or 'z'
True/false	<code>bool</code>	<code>true</code> or <code>false</code>

User-defined:

Real-world concept	C# keyword	Example
Lightweight grouping type	<code>struct</code>	<code>struct Point { public int X, Y; }</code>
Set of named constants	<code>enum</code>	<code>enum Days { Monday, Tuesday }</code>

2.2 Reference types

Predefined:

Real-world concept	C# keyword	Example
Sequence of characters	<code>string</code>	"Hello World!"
Any type at all	<code>object</code>	"Hello" or 5.5

User-defined:

Real-world concept	C# keyword	Example
A constructed object	<code>class</code>	<code>class Dog {}</code>
A contract for objects	<code>interface</code>	<code>interface IDog {}</code>
A reference to a function	<code>delegate</code>	<code>public delegate int Calculate(int x, int y);</code>

Chapter 3

Variables

A variable can be in one of three states:

- Declared without a value: `int a;` Note: some types default to a value (e.g., `int` defaults to 0).
- Assigned a value after declaration: `a = 5;`
- Initialized (declared with a value): `int a = 5;`

Chapter 4

Operators

Operators let us perform calculations on variables or literal values.

4.1 Arithmetic

Name	Operator
Addition	+
Subtraction	-
Multiplication	*
Division	/
Modulus	%
Increment	++
Decrement	--

4.2 Assignment

Name	Operator
Assignment	=
Addition assignment	+=
Subtraction assignment	-=
Multiplication assignment	*=
Division assignment	/=
Modulus assignment	%=

4.3 Comparison

Name	Operator
Equal to	==
Not equal to	!=

Name	Operator
Greater than	>
Less than	<
Greater than or equal to	>=
Less than or equal to	<=

4.4 Logical

Name	Operator
Logical AND	&&
Logical OR	
Logical NOT	!

Chapter 5

Control flow

Control flow statements alter the execution path of a program based on conditions.

5.1 If, if-else, else-if & ternary operator

Used when you have a singular or limited number of conditions to evaluate.

```
if (condition)
{
    // executes if condition is true
}
```

```
if (condition)
{
    // executes if condition is true
}
else
{
    // executes if condition is false
}
```

```
if (condition1)
{
    // executes if condition1 is true
}
else if (condition2)
{
    // executes if condition1 is false and condition2 is true
}
else
{
    // executes if both conditions are false
}
```

The ternary operator is a shorthand for simple if-else assignments:

```
variable = (condition) ? expressionTrue : expressionFalse;
```

5.2 Switch

Used when you have multiple discrete conditions to evaluate. The expression is evaluated once and the matching case executes.

```
switch (expression)
{
    case x:
        // code block
        break;
    case y:
        // code block
        break;
    default:
        // code block
        break;
}
```

5.3 While & do-while loop

Repeats execution while the condition is true. A common idiom for infinite loops is `while (true)`.

The `do-while` variant executes the body at least once, even if the condition is initially false.

```
while (condition)
{
    // code block
}
```

```
do
{
    // code block
}
while (condition);
```

5.4 For & foreach loop

Used when the number of iterations is known ahead of time.

- **Statement 1** — executes once before the loop starts, typically an indexer initialization.
- **Statement 2** — the condition evaluated before each iteration.
- **Statement 3** — executes after each iteration.

```
for (statement 1; statement 2; statement 3)
{
    // code block
}
```


Typical usage:

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

foreach is used to iterate over a collection:

```
foreach (type variableName in collection)
{
    // code block
}
```

5.5 Break & continue

- `break` — exits the loop immediately.
- `continue` — skips the current iteration and moves to the next.

```
while (i < 10)
{
    if (i == 4)
    {
        i++;
        continue;
    }
    Console.WriteLine(i);
    i++;
}
```


Chapter 6

Type conversion and parsing

Conversion is the process of changing a value from one type to another. There are four approaches:

- **Casting** — converts compatible types (e.g., `int` → `double`).
- **as** — safe reference type cast; returns `null` on failure instead of throwing.
- **Conversion methods** — handles more diverse or incompatible types (e.g., `string` → `int`).
- **Parsing** — exclusively converts strings into numeric types.

6.1 Casting compatible types

Implicit (automatic, done by the compiler when safe):

```
int num = 2147483647;
long bigNum = num;
```

Explicit (manual, required when there is potential data loss):

```
double x = 1234.7;
int a = (int)x;

// Derived class to base class
Giraffe g = new Giraffe();
Animal a = g; // class Giraffe : Animal
```

6.2 As conversion

```
object obj = someValue;
SomeType variable = obj as SomeType;

object obj = 123;
string str = obj as string;

if (str == null)
{
```

```
Console.WriteLine("Casting failed!");
}
```

6.3 Conversion methods

- Handles null values gracefully (returns 0 instead of throwing).
- Works across many types including bool, DateTime, etc.

```
string strNum = "42";
int num = Convert.ToInt32(strNum);
```

Method

ToBoolean
 ToByte
 ToChar
 ToDateTime
 ToDecimal
 ToDouble
 ToInt16
 ToInt32
 ToInt64
 ToSbyte
 ToSingle
 ToString
 ToType
 ToUInt16
 ToUInt32
 ToUInt64

6.4 Parsing strings

Parse() — throws an exception if the input is not valid:

```
string strNum = "42";
int num = int.Parse(strNum);
```

TryParse() — safer; returns true on success, false on failure:

```
string strNum = "42";
bool success = int.TryParse(strNum, out int num);
```

Chapter 7

Functions / Methods

A **function** is a named block of code that executes when called. Functions that belong to a class are called **methods**. In C#, every .cs file must contain code inside a class, so every function in C# is a method.

Methods are used to keep code readable, organized, and reusable.

7.1 Working with methods

Every method has:

- A **return type** — what the method returns (void if nothing).
- A **name** (identifier) — its unique name.
- A list of **parameters** (optional) — inputs the caller provides.
- An **access modifier** — controls visibility (public, private, etc.).

```
int Add(int a, int b)
{
    return a + b;
}
```

```
void PrintLine(string sentence)
{
    Console.WriteLine(sentence);
}
```

```
void PrintLine()
{
    Console.WriteLine("I got passed 0 arguments.");
}
```

Calling these methods:

```
int a = Add(5, 5);
PrintLine("int a is equal to: " + a);
PrintLine();
```


Chapter 8

Exception handling

Exception handling prevents programs from crashing unexpectedly when errors occur at runtime.

8.1 What is an exception?

An exception is an error that interrupts normal execution — for example, dividing by zero or accessing an array out of bounds. All exceptions in C# derive from `System.Exception`.

Common built-in exception types:

- `ArithmeticException` — arithmetic errors such as division by zero.
- `FileNotFoundException` — a file was not found at the expected path.
- `IndexOutOfRangeException` — array access outside valid bounds.
- `TimeoutException` — an operation exceeded its time limit.

8.2 Throwing exceptions

```
if (age < 18)
{
    throw new ArithmeticException("Access denied - You must be at least 18 years old.");
}
```

8.3 Handling exceptions

Never let an exception crash the program unexpectedly unless absolutely necessary. Use `try-catch` to handle them.

```
try
{
    int number = int.Parse("NotANumber");
}
catch (FormatException ex)
{
}
```

```
    Console.WriteLine("Error: " + ex.Message);  
}
```

The `finally` block always runs regardless of whether an exception was thrown — use it to release resources:

```
try  
{  
    Console.WriteLine("Opening file...");  
}  
catch (Exception ex)  
{  
    Console.WriteLine("Error: " + ex.Message);  
}  
finally  
{  
    Console.WriteLine("Cleanup: Closing file...");  
}
```

8.4 Creating user-defined exceptions

1. Inherit from `System.Exception`.
2. Add `[Serializable]` attribute.
3. Provide the standard exception constructors.

```
[Serializable]  
public class MyException : Exception  
{  
    public MyException() {}  
  
    public MyException(string message)  
        : base(message) {}  
  
    public MyException(string message, Exception innerException)  
        : base(message, innerException) {}  
}
```


Chapter 9

Preprocessor directives

Preprocessor directives are instructions to the compiler that control how source code is processed before compilation. They do not produce executable code — think of them as the control flow of the compilation step, not of the program itself.

Directive	Description
<code>#define</code>	Defines a symbol (not a variable).
<code>#undef</code>	Undefines a previously defined symbol.
<code>#if</code> , <code>#elif</code> , <code>#else</code> , <code>#endif</code>	Conditionally includes or excludes code.
<code>#region</code> , <code>#endregion</code>	Organizes code into collapsible sections in the IDE.
<code>#error</code>	Forces a compilation error with a custom message.
<code>#warning</code>	Issues a compiler warning with a custom message.
<code>#pragma</code>	Provides special compiler instructions (e.g., disabling specific warnings).
<code>#line</code>	Modifies the line numbers reported by the compiler for debugging.

```
class Program
{
    #region Initialization
    static void Initialize()
    {
        Console.WriteLine("Initializing...");
    }
    #endregion

    #region Cleanup
    static void Cleanup()
    {
        Console.WriteLine("Cleaning up...");
    }
}
```

```
}  
#endregion  
}
```

Chapter 10

Data collections

Data collections store and manage multiple values in a single structure. Each type serves a different purpose depending on how data needs to be accessed or modified.

10.1 Summary

Collection	Description	When to use
Array	Fixed-size, indexed	Size is known and unchanging
List	Dynamic-size, indexed	Size may change dynamically
Dictionary	Key-value pairs, fast lookup	Fast access by unique key
Enum	Named constants	Related constant values
Queue	FIFO	Queue-like processing
Stack	LIFO	Stack-like processing
HashSet	Unordered, unique elements	Duplicate prevention
SortedList	Key-value pairs, sorted by key	Sorted key-value access

10.2 Arrays

Fixed-size collection of same-type elements.

- Fixed size — cannot change after creation.
- Indexed — 0-based access.
- Homogeneous — all elements must be the same type.

```
int[] numbers = new int[3];
numbers[0] = 1;
numbers[1] = 2;
numbers[2] = 3;

Console.WriteLine(numbers[0]); // Output: 1
```

10.3 Enums

A special type representing a set of named constants.

- Named values — replace magic numbers with readable names.
- Underlying type is `int` by default, but can be changed.

```
enum Days { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday }

Days today = Days.Monday;
Console.WriteLine(today); // Output: Monday

int dayNumber = (int)Days.Wednesday;
Console.WriteLine(dayNumber); // Output: 3
```

10.4 Lists

Dynamic collection from `System.Collections.Generic`.

- Flexible size — grows and shrinks as needed.
- Indexed — 0-based access.
- Better than arrays when resizing is frequent.

```
List<int> numbers = new List<int>();
numbers.Add(1);
numbers.Add(2);
numbers.Add(3);

Console.WriteLine(numbers[0]); // Output: 1
numbers.Remove(2);
```

10.5 Dictionaries

Key-value pair collection from `System.Collections.Generic`.

- Each key is unique.
- Keys are hashed for fast lookup.
- No fixed size.

```
Dictionary<string, int> phoneBook = new Dictionary<string, int>();
phoneBook.Add("Alice", 123456789);
phoneBook.Add("Bob", 987654321);

Console.WriteLine(phoneBook["Alice"]); // Output: 123456789
```

10.6 Queues

First-In, First-Out (FIFO) collection. Elements added at the back, removed from the front.

```
Queue<string> queue = new Queue<string>();
queue.Enqueue("Task 1");
queue.Enqueue("Task 2");

Console.WriteLine(queue.Dequeue()); // Output: Task 1
```

10.7 Stacks

Last-In, First-Out (LIFO) collection. Elements added and removed from the top.

```
Stack<string> stack = new Stack<string>();
stack.Push("Task 1");
stack.Push("Task 2");

Console.WriteLine(stack.Pop()); // Output: Task 2
```

10.8 HashSet

Unordered collection that only allows unique elements.

- Automatically rejects duplicates.
- No guaranteed order.

```
HashSet<int> uniqueNumbers = new HashSet<int>();
uniqueNumbers.Add(1);
uniqueNumbers.Add(2);
uniqueNumbers.Add(2); // Duplicate, not added

Console.WriteLine(uniqueNumbers.Count); // Output: 2
```

10.9 SortedList

Key-value pairs kept in sorted key order. Uses binary search for efficient lookup.

```
SortedList<int, string> sortedList = new SortedList<int, string>();
sortedList.Add(3, "Three");
sortedList.Add(1, "One");
sortedList.Add(2, "Two");

Console.WriteLine(sortedList[1]); // Output: One
```


Chapter 11

Object-Oriented Programming (OOP)

“An object is an instance of a user-defined data type that also holds the procedures (methods) that operate on its data.”

Before diving deeper, compare OOP to the alternative approach:

Procedural Programming	Object-Oriented Programming
Focuses on functions that operate on data.	Focuses on objects containing both data and behavior.
Data is separate from logic.	Data and logic are bundled together.
Example: C.	Example: C#, Java, Python.
Less structured for large projects.	Easier to scale and maintain.

Procedural example:

```
string name = "Alice";

void SayHello(string name)
{
    Console.WriteLine("Hello, " + name);
}

SayHello(name);
```

OOP example:

```
class Person
{
    public string Name;

    public void SayHello()
    {
        Console.WriteLine("Hello, " + Name);
    }
}
```

```
Person person = new Person();
person.Name = "Alice";
person.SayHello();
```

11.1 Why OOP?

- Clear, organized structure.
- Prevents code duplication (DRY — Don't Repeat Yourself).
- Easier to maintain, scale, modify, and debug.
- Promotes reusable, modular components.

11.2 The four pillars of OOP

11.2.1 1. Encapsulation — keep data safe inside objects

Restricts direct access to an object's internal data. Prevents accidental modification and enforces controlled access, typically through **properties**.

```
class Person
{
    // Private field - cannot be accessed directly
    private int age;

    // Public property with validation
    public int Age
    {
        get { return age; }
        set
        {
            if (value >= 0)
                age = value;
            else
                Console.WriteLine("Age cannot be negative!");
        }
    }
}
```

Properties are the standard mechanism for encapsulation:

```
private string name; // backing field

public string Name    // property
{
    get { return name; }
    set { name = value; }
}
```


11.2.2 2. Abstraction — hide unnecessary details

Exposes only what is essential to the caller. Internal implementation details are hidden.

```
class Car
{
    public void Start()
    {
        StartIgnition(); // hidden
        InjectFuel();     // hidden
        StartEngine();    // hidden
        Console.WriteLine("Car started!");
    }

    private void StartIgnition() { /* ... */ }
    private void InjectFuel()    { /* ... */ }
    private void StartEngine()   { /* ... */ }
}
```

Abstraction is enforced through access modifiers:

Modifier	Description
public	Accessible by all classes.
private	Accessible only within the same class.
protected	Accessible within the same class and derived classes.
internal	Accessible within the same assembly only.

Combinations: protected internal and private protected.

11.2.3 3. Inheritance — reuse and extend existing code

Allows a child class to inherit fields and methods from a parent class, reducing duplication.

```
class Vehicle // Base class
{
    public void Drive() { Console.WriteLine("Vehicle is driving."); }
}

class Car : Vehicle { }
class Truck : Vehicle { }

Car myCar = new Car();
Truck myTruck = new Truck();
myCar.Drive(); // Output: "Vehicle is driving."
myTruck.Drive(); // Output: "Vehicle is driving."
```

11.2.4 4. Polymorphism — one interface, many implementations

The same method can behave differently depending on the object invoking it.

```
class Animal
{
    public virtual void MakeSound() { Console.WriteLine("Some sound..."); }
}

class Bird : Animal
{
    public override void MakeSound() { Console.WriteLine("Chirp Chirp!"); }
}

class Dog : Animal
{
    public override void MakeSound() { Console.WriteLine("Woof Woof!"); }
}

Animal myBird = new Bird();
Animal myDog = new Dog();
myBird.MakeSound(); // Output: "Chirp Chirp!"
myDog.MakeSound();  // Output: "Woof Woof!"
```

Interfaces are another form of polymorphism — completely abstract types that define only method signatures. They act as a contract: you define what an object must be able to do, but the implementation is left to the class.

By convention, interface names start with I.

```
interface IAnimal
{
    void AnimalSound();
}

class Pig : IAnimal
{
    public void AnimalSound()
    {
        Console.WriteLine("The pig says: wee wee");
    }
}

class Program
{
    static void Main(string[] args)
    {
        IAnimal myPig = new Pig();
        myPig.AnimalSound();
    }
}
```

Chapter 12

Nullable value types

Value types (`int`, `float`, `bool`, etc.) cannot hold `null` by default. To allow them to represent the absence of a value — useful for optional fields or database results — use the `?` suffix:

```
int? age = null;
double? price = 99.99;
bool? isAvailable = null;
```

12.1 How it works internally

`T?` is syntactic sugar for `Nullable<T>`, a struct with two key members:

- `HasValue` — `true` if a value is present.
- `Value` — the actual value if `HasValue` is `true`.

```
[Serializable]
public struct Nullable<T> where T : struct
{
    public bool HasValue { get; }
    public T Value { get; }
}
```

`int? i = null;` is equivalent to `Nullable<int> i = null;`.

12.2 Null-coalescing operator (??)

Provides a fallback value when the nullable is `null`:

```
int? count = null;

// Assigns 0 if count is null
int result = count ?? 0;
Console.WriteLine(result); // Output: 0
```

12.3 Null-conditional operator (?.)

Safely accesses members on a potentially null value — no exception if null:

```
int? length = null;  
Console.WriteLine(length?.ToString()); Prints nothing, no exception
```

12.4 Converting nullable to non-nullable

Assigning a nullable to a non-nullable requires explicit null handling:

```
int? nullableNumber = null;  
  
// Assigns -1 if null  
int nonNullableNumber = nullableNumber ?? -1;
```

Chapter 13

Pass by reference

The `ref` and `out` keywords pass arguments by reference. Any changes made to these parameters inside the method are reflected in the original variable at the call site.

13.1 `ref`

The variable must be initialized before being passed. Allows both reading and writing inside the method.

```
public static void ModifyValue(ref int number)
{
    number += 10;
}

public static void Main()
{
    int x = 5;
    Console.WriteLine("Before: " + x); // Output: Before: 5
    ModifyValue(ref x);
    Console.WriteLine("After: " + x);  // Output: After: 15
}
```

13.2 `out`

The variable does not need to be initialized before being passed. The method must assign a value before returning. Commonly used to return multiple values from a method.

```
public static void Divide(int numerator, int denominator, out int result)
{
    if (denominator == 0)
        result = 0;
    else
        result = numerator / denominator;
}
```

```
public static void Main()
{
    int result;
    Divide(10, 2, out result);
    Console.WriteLine("Result: " + result); // Output: Result: 5

    Divide(10, 0, out result);
    Console.WriteLine("Result: " + result); // Output: Result: 0
}
```

Chapter 14

Tuples

A data structure that holds a fixed sequence of elements of potentially different types. Supports 1 to 8 elements; exceeding 8 causes a compiler error.

```
Tuple<int, string, string> person = new Tuple<int, string, string>(1, "Steve", "Jobs");
```

Named tuple elements (preferred, more readable):

```
(int Id, string FirstName, string LastName) person = (1, "Bill", "Gates");  
Console.WriteLine(person.Id);           // 1  
Console.WriteLine(person.FirstName);    // Bill  
Console.WriteLine(person.LastName);     // Gates
```

Returning a tuple from a method:

```
public (string, int) GetStringAndInt()  
{  
    return ("Hello", 42);  
}
```


Chapter 15

Delegates

A delegate is a **type-safe function pointer**, similar to function pointers in C/C++. It stores a reference to a method and allows calling that method dynamically.

Three steps:

1. Declare a delegate type (defines the method signature).
2. Assign a method to the delegate.
3. Invoke the delegate.

```
// 1. Declare
public delegate void MyDelegate(string msg);

// 2. Target method - signature must match the delegate
static void MethodA(string message)
{
    Console.WriteLine(message);
}

// 3. Assign and invoke
MyDelegate del = new MyDelegate(MethodA);
// or
MyDelegate del = MethodA;
// or via lambda
MyDelegate del = (string msg) => Console.WriteLine(msg);

del.Invoke("Hello World!");
// or
del("Hello World!");
```

15.1 Passing delegates

A delegate can be passed as an argument to another method:

```
static void InvokeDelegate(MyDelegate del)
{
    del("Hello World");
}
```

15.2 Multicast delegates

A delegate can point to multiple methods simultaneously. All methods in the invocation list are called in order when the delegate is invoked.

- `+` / `+=` — add a method to the invocation list.
- `-` / `-=` — remove a method from the invocation list.

```
MyDelegate del1 = ClassA.MethodA;
MyDelegate del2 = ClassB.MethodB;
MyDelegate del = del1 + del2;           // del1 + del2
MyDelegate del3 = msg => Console.WriteLine("Lambda: " + msg);
del += del3;                           // del1 + del2 + del3
del -= del2;                           // removes del2
del -= del1;                           // removes del1
```

15.3 Generic delegates

```
public delegate T Add<T>(T param1, T param2);
```

15.4 Built-in generic delegates: Func, Action, Predicate

These eliminate the need to declare a custom delegate for common patterns:

- **Func** — zero or more parameters, returns a value.
- **Action** — zero or more parameters, returns void.
- **Predicate** — one parameter, returns bool.

```
Func<int, int, int> add = (a, b) => a + b;
Action<string> print = message => Console.WriteLine(message);
Predicate<int> isEven = num => num % 2 == 0;
```

15.5 Anonymous methods

An anonymous method is defined inline inside a delegate without a separate named method:

```
public delegate void Greet(string name);

static void Main()
{
    Greet greet = delegate (string name)
    {
```

```
        Console.WriteLine("Hello, " + name);  
    };  
  
    greet("Alice"); // Output: Hello, Alice  
}
```

Limitations of anonymous methods:

- Not reusable.
- No `goto`, `break`, or `continue`.
- No `ref` or `out` parameters from an outer method.
- Cannot access unsafe code.
- Cannot appear on the left side of the `is` operator.

15.6 Use cases

- Event handling (e.g., button click in UI).
- Callbacks — execute a method after another completes.
- LINQ and functional-style programming.
- Strategy pattern and runtime logic switching.

Chapter 16

Lambda expressions

A lambda expression is a shorthand for writing small, anonymous functions.

Instead of:

```
int MultiplyByFive(int num)
{
    return num * 5;
}
```

Write:

```
num => num * 5;
```

A lambda does not execute on its own — it must be assigned to a delegate or used inside another method:

```
Func<int, int> multiply = num => num * 5;
Console.WriteLine(multiply(3)); // Output: 15

Action<string> greet = name => Console.WriteLine("Hello, " + name);
greet("Alice"); // Output: Hello, Alice
```

16.1 Two types of lambdas

Expression lambda — single expression body, implicit return:

```
(num) => num * 5;
```

Statement lambda — multiple statements, requires explicit `return`:

```
(num) =>
{
    int result = num * 5;
    return result;
};
```

Feature	Expression lambda	Statement lambda
Syntax	Simple, single expression	Supports multiple statements
Implicit return	Yes	No — explicit return required
Best for	Short, single-line logic	Complex, multi-line logic

Syntax recap: `(parameterList) => lambdaBody`

Chapter 17

Expression-bodied members

A shorter syntax for methods, properties, and other members that consist of a single expression. Uses => (fat arrow) instead of { }.

Lambda:

```
(int x, int y) => x + y;
```

Method:

```
public int Add(int x, int y) => x + y;
```

Property (read-only):

```
public int MyProperty => 42;
```

Indexer:

```
public int this[int index] => index * 2;
```

Constructor:

```
public MyClass(string name) => Name = name;

// Equivalent to:
public MyClass(string name)
{
    Name = name;
}
```


Chapter 18

Generics

Generics allow a class or method to work with any data type, determined at the point of use. Comparable to templates in C++.

```
// Generic class
class DataStore<T>
{
    public T Data { get; set; }

    public void Print<T>(T data)
    {
        Console.WriteLine(data);
    }
}

// Instantiating with a specific type
DataStore<string> dataStore = new DataStore<string>();

// Calling a generic method
dataStore.Print<int>(100);
```

T is a convention — the type parameter can be named anything.

Chapter 19

Extension methods

Extension methods add new methods to existing types without modifying their source code. Useful when you cannot change a class (e.g., types from external libraries or the .NET framework itself).

Rules:

- Must be defined in a **static** class.
- The first parameter must be the type being extended, prefixed with **this**.

```
namespace ExtensionMethods
{
    public static class IntExtensions
    {
        public static bool IsGreaterThan(this int i, int value)
        {
            return i > value;
        }
    }
}

// Usage:
int i = 10;
bool result = i.IsGreaterThan(100); // false
```


Chapter 20

Indexers

An indexer allows an object to be accessed using `[]` syntax, as if it were an array, even when it is a custom class or struct.

- Works like a property but accepts an index parameter.
- Can wrap any internal collection (array, list, dictionary, etc.).

```
public class MyClass
{
    private int[] numbers = { 1, 2, 3, 4, 5 };

    public int this[int index]
    {
        get { return numbers[index]; }
        set { numbers[index] = value; }
    }
}
```


Chapter 21

Operator overloading

Operator overloading allows you to redefine how built-in operators behave for user-defined types. You specify the behavior using the `operator` keyword.

```
public static Box operator+(Box b, Box c)
{
    Box box = new Box();
    box.length = b.length + c.length;
    box.breadth = b.breadth + c.breadth;
    box.height = b.height + c.height;
    return box;
}
```


Chapter 22

Partials

`partial` allows a class or method definition to be split across multiple files. At compile time, all parts are merged into a single definition.

22.1 Partial classes

Common uses:

- Large classes split across multiple developers.
- Auto-generated code (WPF, WinForms, Blazor code-behind).
- Organizing related methods into separate files under the same class.

Person.cs:

```
public partial class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }

    public void PrintFullName()
    {
        Console.WriteLine($"{FirstName} {LastName}");
    }
}
```

Person_2.cs:

```
public partial class Person
{
    public int Age { get; set; }

    public void PrintAge()
    {
        Console.WriteLine($"Age: {Age}");
    }
}
```

22.2 Partial methods

A partial method is declared in one part and optionally implemented in another. If no implementation is provided, the compiler removes the call entirely — no runtime overhead.

Constraints:

- Must be inside a partial class.
- Must return `void`.
- Cannot have access modifiers.
- Cannot be `virtual`, `abstract`, or `override`.
- Must use `partial` in both declaration and implementation.

User_Declaration.cs:

```
public partial class User
{
    public string Name { get; set; }

    partial void OnNameChanged();

    public void ChangeName(string newName)
    {
        Name = newName;
        OnNameChanged();
    }
}
```

User_Definition.cs:

```
public partial class User
{
    partial void OnNameChanged()
    {
        Console.WriteLine($"Name changed to: {Name}");
    }
}
```

Chapter 23

Method hiding

When a derived class defines a method with the same name as a base class method **without using virtual/override**, the derived method hides the base method. Method resolution is based on the reference type, not the actual object type.

```
class Person
{
    public void Greet()
    {
        Console.WriteLine("Hi! I am a person.");
    }
}

class Employee : Person
{
    public void Greet()
    {
        Console.WriteLine("Hello! I am an employee.");
    }
}

class Program
{
    static void Main()
    {
        Person p1 = new Person();

        // Calls Person.Greet()
        p1.Greet();

        // Still calls Person.Greet()
        // Reference type determines the call
        Person p2 = new Employee();
        p2.Greet();
    }
}
```

```
Employee emp = new Employee();

// Calls Employee.Greet()
emp.Greet();
}
}
```

Output:

```
Hi! I am a person.
Hi! I am a person.
Hello! I am an employee.
```

23.1 Intentional hiding with the new keyword

Use `new` to explicitly signal to the compiler that the hiding is intentional — suppresses the compiler warning:

```
class Employee : Person
{
    public new void Greet()
    {
        Console.WriteLine("I am the Manager!");
    }
}
```

Key point: even with `new`, a base class reference (`Person p2 = new Employee()`) still calls `Person.Greet()`. The actual object type only matters when accessed through a derived class reference.

Chapter 24

Events

An event is a notification sent by an object to signal that something has occurred. In C#, an event is an **encapsulated delegate** — it depends on a delegate for its signature.

24.1 Observer design pattern

- The class that raises events is the **Publisher**.
- The class that receives notifications is the **Subscriber**.
- Multiple subscribers can register for the same event.

24.2 Declaring an event

Two steps:

1. Declare a delegate.
2. Declare an event variable of that delegate type using the **event** keyword.

24.2.1 Publisher class

```
public delegate void Notify();

public class ProcessBusinessLogic
{
    public event Notify ProcessCompleted;

    public void StartProcess()
    {
        Console.WriteLine("Process Started!");
        OnProcessCompleted();
    }

    protected virtual void OnProcessCompleted()
    {
        // ? means: only invoke if not null
    }
}
```

```
        ProcessCompleted?.Invoke();
    }
}
```

24.2.2 Subscriber class

```
class Program
{
    public static void Main()
    {
        ProcessBusinessLogic bl = new ProcessBusinessLogic();

        // subscribe
        bl.ProcessCompleted += bl_ProcessCompleted;
        bl.StartProcess();
    }

    public static void bl_ProcessCompleted()
    {
        Console.WriteLine("Process Completed!");
    }
}
```

24.2.3 Simplified example

```
public class Subject
{
    public delegate void Notify();
    public event Notify Observers;

    public void ChangeState()
    {
        Console.WriteLine("State changed!");
        Observers?.Invoke();
    }
}

public class Observer
{
    public void OnStateChange() => Console.WriteLine("Observer notified!");
}
```

24.3 Built-in EventHandler delegate

.NET provides `EventHandler` and `EventHandler<TEventArgs>` for the most common event patterns. Use `EventHandler` when no event data needs to be passed:

```
public event EventHandler ProcessCompleted;

public void StartProcess()
{
    Console.WriteLine("Process Started!");
    OnProcessCompleted(EventArgs.Empty);
}

protected virtual void OnProcessCompleted(EventArgs e)
{
    ProcessCompleted?.Invoke(this, e);
}
```

24.4 Passing event data

Derive a custom class from `EventArgs` to carry data with the event. Convention: class name ends with `EventArgs`.

```
class ProcessEventArgs : EventArgs
{
    public bool IsSuccessful { get; set; }
    public DateTime CompletionTime { get; set; }
}
```

Passing data via `EventHandler<T>`:

```
public class ProcessBusinessLogic
{
    public event EventHandler<bool> ProcessCompleted;

    public void StartProcess()
    {
        try
        {
            Console.WriteLine("Process Started!");
            OnProcessCompleted(true);
        }
        catch (Exception ex)
        {
            OnProcessCompleted(false);
        }
    }

    protected virtual void OnProcessCompleted(bool isSuccessful)
    {
        ProcessCompleted?.Invoke(this, isSuccessful);
    }
}
```

```
class Program
{
    public static void Main()
    {
        ProcessBusinessLogic bl = new ProcessBusinessLogic();
        bl.ProcessCompleted += bl_ProcessCompleted;
        bl.StartProcess();
    }

    public static void bl_ProcessCompleted(object sender, bool isSuccessful)
    {
        Console.WriteLine("Process " + (isSuccessful ? "Completed Successfully" : "failed"));
    }
}
```


Chapter 25

Attributes

Attributes are special metadata markers that can be attached to classes, methods, properties, and other code elements. They do not change runtime behavior directly but provide information to the compiler, runtime, or other tools.

Think of them as labels or annotations on code elements — added using square brackets []:

```
[Obsolete("This method is outdated, use NewMethod instead")]
void OldMethod()
{
    Console.WriteLine("This is old.");
}
```

25.1 Predefined attributes

Obsolete — marks something as outdated, optionally treating usage as an error:

```
// true = compile error
[Obsolete("Use NewMethod instead", true)]
void OldMethod() {}
```

Conditional — method only executes when a specific symbol is defined (e.g., DEBUG builds):

```
#define DEBUG
using System.Diagnostics;

public class MyClass
{
    [Conditional("DEBUG")]
    public static void DebugMessage(string msg)
    {
        Console.WriteLine(msg);
    }
}
```

AttributeUsage — controls where a custom attribute can be applied:

```
[AttributeUsage(AttributeTargets.Method | AttributeTargets.Class)]
class MyCustomAttribute : Attribute {}
```

25.2 User-defined attributes

Create custom attributes by inheriting from `System.Attribute`:

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method, AllowMultiple = true)]
public class BugInfo : Attribute
{
    public int BugID { get; }
    public string Developer { get; }
    public string LastReview { get; }

    public BugInfo(int bugID, string developer, string lastReview)
    {
        BugID = bugID;
        Developer = developer;
        LastReview = lastReview;
    }
}

[BugInfo(101, "Alice", "2024-03-06")]
class MyProgram {}

[BugInfo(202, "Bob", "2024-02-28")]
void FixBug() {}
```

Reading attributes at runtime using Reflection:

```
Type type = typeof(MyProgram);
foreach (var attr in type.GetCustomAttributes(false))
{
    BugInfo bug = (BugInfo)attr;
    Console.WriteLine($"Bug {bug.BugID}, Reported by {bug.Developer}, Last Reviewed: {bug.LastReview}");
}
```

Chapter 26

Reflection

Reflection, found in `System.Reflection`, allows a program to inspect and interact with its own metadata at runtime — examining types, methods, and properties without knowing them at compile time.

Use cases:

- Read attribute information at runtime.
- Examine types inside an assembly (`.exe` or `.dll`).
- Late binding — invoke methods on objects whose types are unknown at compile time.
- Dynamically create and use types (used heavily in DI frameworks).

Define a custom attribute:

```
[AttributeUsage(AttributeTargets.Class)]
public class HelpAttribute : Attribute
{
    public string Url { get; }

    public HelpAttribute(string url)
    {
        Url = url;
    }
}

[Help("https://docs.microsoft.com/en-us/dotnet/csharp/")]
class MyClass { }
```

Read it at runtime using Reflection:

```
using System.Reflection;

class Program
{
    static void Main()
    {
        Type type = typeof(MyClass);
```

```
object[] attributes = type.GetCustomAttributes(false);

foreach (var attribute in attributes)
{
    if (attribute is HelpAttribute help)
    {
        Console.WriteLine($"Help URL: {help.Url}");
    }
}
}
```

Output: Help URL: <https://docs.microsoft.com/en-us/dotnet/csharp/>

Chapter 27

Language Integrated Query (LINQ)

LINQ lets you query collections (arrays, lists, databases, XML, etc.) using a readable, expressive syntax directly in C#.

Query syntax:

```
int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

var evens = from num in numbers
            where num % 2 == 0
            select num;

foreach (var num in evens)
    Console.WriteLine(num);
```

Method syntax (equivalent):

```
var evens = numbers.Where(num => num % 2 == 0);
```

Example with objects:

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

var people = new List<Person>
{
    new Person { Name = "Alice", Age = 25 },
    new Person { Name = "Bob", Age = 30 },
    new Person { Name = "Charlie", Age = 20 }
};

var result = people.Where(p => p.Age > 25)
                   .OrderBy(p => p.Age)
                   .Select(p => p.Name);
```

```
foreach (var name in result)
    Console.WriteLine(name);
```

27.1 LINQ methods

Method	Description	Example
Where	Filters elements	<code>list.Where(x => x > 5)</code>
Select	Transforms each element	<code>list.Select(x => x * 2)</code>
OrderBy	Ascending sort	<code>list.OrderBy(x => x)</code>
OrderByDescending	Descending sort	<code>list.OrderByDescending(x => x)</code>
First / FirstOrDefault	First element	<code>list.FirstOrDefault()</code>
Last / LastOrDefault	Last element	<code>list.LastOrDefault()</code>
Count	Element count	<code>list.Count()</code>
Sum	Sum of values	<code>list.Sum()</code>

Chapter 28

Multithreading / Asynchronous programming

Normally C# runs code line by line (synchronously). Multithreading allows multiple operations to run concurrently, improving performance.

Key distinction:

- **Multithreading** — multiple threads running simultaneously.
- **Async programming** — a single thread handles multiple tasks by switching between them instead of blocking.

When a `.exe` runs, Windows creates a process. Execution happens inside threads owned by that process.

Two approaches:

- `System.Threading` — low-level thread control.
- `System.Threading.Tasks` — preferred for modern async code.

28.1 Threads (`System.Threading`)

```
using System.Threading;

class Program
{
    public static void Main()
    {
        Thread mainThread = Thread.CurrentThread;
        mainThread.Name = "___MAIN__THREAD__";

        Thread thread1 = new Thread(CountDown);
        Thread thread2 = new Thread(CountUp);

        thread1.Start();
        thread2.Start();
    }
}
```

```
        Console.WriteLine(mainThread.Name + " is complete!");
    }

    public static void CountDown()
    {
        for (int i = 10; i >= 0; i--)
            Console.WriteLine($"Counting Down: {i}");
    }

    public static void CountUp()
    {
        for (int i = 0; i <= 10; i++)
            Console.WriteLine($"Counting Up: {i}");
    }
}
```

28.2 Tasks (System.Threading.Tasks)

Task is the modern alternative to Thread. More efficient, integrates with `async/await`.

```
using System.Threading.Tasks;

class Program
{
    public static async Task Main()
    {
        Task task1 = Task.Run(() => CountDown());
        Task task2 = Task.Run(() => CountUp());

        await Task.WhenAll(task1, task2);
    }

    public static void CountDown()
    {
        for (int i = 10; i >= 0; i--)
            Console.WriteLine($"Counting Down: {i}");
    }

    public static void CountUp()
    {
        for (int i = 0; i <= 10; i++)
            Console.WriteLine($"Counting Up: {i}");
    }
}
```

Note: When using Tasks, avoid mixing Thread objects within task callbacks (e.g., `Thread.Sleep(500)`). Use `Task.Delay(500)` instead to avoid issues.

Chapter 29

The IDisposable interface

IDisposable allows an object to explicitly release unmanaged resources (file handles, database connections, network sockets, etc.) when it is no longer needed.

The C# Garbage Collector (GC) automatically cleans up managed memory but does **not** clean up unmanaged resources. IDisposable fills that gap by exposing a `Dispose()` method.

29.1 How to implement IDisposable

```
class MyResource : IDisposable
{
    private bool _disposed = false;

    public void UseResource()
    {
        if (_disposed) throw new ObjectDisposedException(nameof(MyResource));
        Console.WriteLine("Using resource...");
    }

    public void Dispose()
    {
        if (!_disposed)
        {
            Console.WriteLine("Releasing resource...");
            _disposed = true;
        }
    }
}
```

using statement (best practice) — automatically calls `Dispose()` even if an exception is thrown:

```
using (MyResource resource = new MyResource())
{
    resource.UseResource();
} // Dispose() called automatically here
```

Manual disposal:

```
MyResource resource = new MyResource();
resource.UseResource();
resource.Dispose();
```

29.2 Real-world example — SqlConnection wrapper

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Configuration;

namespace DataVista.Database
{
    public class dvConnection : IDisposable
    {
        private bool _disposed = false;
        private SqlConnection? _sqlConnection;
        private string _connectionString = ConfigurationManager.ConnectionStrings["DbConnection"]

        public dvConnection(SqlConnection sqlConnection)
        {
            _sqlConnection = sqlConnection ?? throw new DataException($"{this} failed to initia

        public dvConnection(string connectionString)
        {
            _sqlConnection = new SqlConnection(connectionString);
        }

        ~dvConnection()
        {
            _disposed = true;
            _sqlConnection?.Dispose();
            Dispose(true);
        }

        public SqlConnection SqlConnection => _sqlConnection;
        public string ConnectionString => _connectionString;

        public void Dispose()
        {
            _disposed = true;
            GC.SuppressFinalize(this);
        }
    }
}
```

```
protected virtual void Dispose(bool disposing)
{
    if (!_disposed)
    {
        if (disposing)
        {
            _sqlConnection?.Dispose();
            _sqlConnection = null;
        }
        _disposed = true;
    }
}

public override string ToString() => $"Connection is {_sqlConnection?.State}.";
}
}
```


Chapter 30

Unsafe

The `unsafe` keyword enables pointer-based code and direct memory access. By default, C# runs in a safe managed environment that prevents raw memory manipulation. `unsafe` lifts those restrictions, allowing C/C++-style pointer operations.

Typical uses: performance-critical code, interop with unmanaged libraries, low-level system programming.

Key points:

- **Pointer declaration** — `int* p`; stores the address of a variable.
- **unsafe block** — code using pointers must be inside an `unsafe` block or method.
- **fixed keyword** — pins a managed object in memory so a pointer can safely reference it (prevents the GC from moving it).
- **Compilation** — the `/unsafe` compiler option or `<AllowUnsafeBlocks>true</AllowUnsafeBlocks>` in the project file must be enabled.

```
unsafe
{
    int var = 20;
    int* p = &var;
    Console.WriteLine("Data: {0}, Address: {1}", var, (int)p);
}
```


Chapter 31

Dependency Injection (DI)

Dependency Injection is a technique for providing objects (dependencies) to a class from outside rather than having the class create them internally. This reduces tight coupling between components.

Not to be confused with WPF's `dependency properties`.

Benefits:

- **Loose coupling** — classes depend on abstractions (interfaces), not concrete implementations.
- **Testability** — dependencies can be mocked for unit testing.
- **Maintainability** — swap implementations without modifying dependent classes.
- **Reusability** — components can be used across different applications.

31.1 Constructor injection

Dependencies provided through the constructor — the most common and recommended form:

```
public interface IMessageService
{
    void SendMessage(string message);
}

public class EmailService : IMessageService
{
    public void SendMessage(string message)
    {
        Console.WriteLine($"Email sent: {message}");
    }
}

public class Notification
{
    private readonly IMessageService _messageService;

    public Notification(IMessageService messageService)
    {

```

```
        _messageService = messageService;
    }

    public void Notify(string message)
    {
        _messageService.SendMessage(message);
    }
}

// Usage:
IMessageService emailService = new EmailService();
Notification notification = new Notification(emailService);
notification.Notify("Hello via Email!");
```

31.2 Property injection

Dependencies set via public properties after construction:

```
public class Notification
{
    public IMessageService MessageService { get; set; }

    public void Notify(string message)
    {
        MessageService?.SendMessage(message);
    }
}

// Usage:
var notification = new Notification();
notification.MessageService = new EmailService();
notification.Notify("Hello via Email!");
```

31.3 Method injection

Dependency passed directly as a method parameter:

```
public class Notification
{
    public void Notify(string message, IMessageService messageService)
    {
        messageService.SendMessage(message);
    }
}

// Usage:
Notification notification = new Notification();
```



```
notification.Notify("Hello via Email!", new EmailService());
```

31.4 Using an IoC container

In real projects, dependencies are wired up using an IoC (Inversion of Control) container rather than manually. The built-in .NET DI container is the standard choice:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddScoped<IMessageService, EmailService>();
    services.AddScoped<Notification>();
}
```


Chapter 32

WPF Dependency Properties (DP)

A Dependency Property is an advanced property system used in Windows Presentation Foundation (WPF) that **stores values in a centralized property store** rather than in a private backing field. This enables features that regular C# properties cannot support.

Not to be confused with **Dependency Injection**.

Dependency properties enable:

- Data binding — bind UI elements to data sources.
- Styles and triggers — modify properties dynamically.
- Animations — WPF animations can target dependency properties.
- Value inheritance — values can propagate down the visual tree.

32.1 Creating a dependency property

A dependency property must be registered in a class that inherits from `DependencyObject`:

```
using System.Windows;
using System.Windows.Controls;

public class MyCustomControl : Control
{
    // Register the dependency property
    public static readonly DependencyProperty MyTextProperty =
        DependencyProperty.Register(
            "MyText",                // Property name
            typeof(string),          // Property type
            typeof(MyCustomControl), // Owner class
            new PropertyMetadata("Default Value") // Default value
        );

    // CLR wrapper
    public string MyText
    {
        get { return (string)GetValue(MyTextProperty); }
    }
}
```

```
        set { SetValue(MyTextProperty, value); }  
    }  
}
```

32.2 Attached dependency properties

An Attached Property is declared in one class but usable by other controls. The canonical example is `Grid.Column`:

```
<Grid>  
    <Button Grid.Column="0">Click</Button>  
    <Button Grid.Column="1">Clack</Button>  
</Grid>
```

`Grid.Column` is not defined on `Button`, but the `Grid` class registers it as an attached property that any element can use:

```
public class Grid  
{  
    public static readonly DependencyProperty ColumnProperty =  
        DependencyProperty.RegisterAttached(  
            "Column",  
            typeof(int),  
            typeof(Grid)  
        );  
  
    public static int GetColumn(DependencyObject obj) => (int)obj.GetValue(ColumnProperty);  
    public static void SetColumn(DependencyObject obj, int value) => obj.SetValue(ColumnProperty, value);  
}
```

Chapter 33

ASP.NET Core Web API

ASP.NET Core provides a framework for building APIs using the MVC (Model-View-Controller) pattern. REST is an architectural style — not a standard — that defines how a well-structured API should behave.

33.1 REST constraints

- Uniform Interface
- Client-server separation
- Statelessness
- Layered system
- Cacheability
- Code on demand (optional)

API maturity is measured by the Richardson Maturity Model.

33.2 HTTP verbs and CRUD mapping

CRUD operation	HTTP verb
Create	POST
Read	GET
Update	PUT
Delete	DELETE

Additional verbs:

- PATCH — partial update of a resource.
- HEAD — same as GET but returns headers only, no body.
- OPTIONS — returns supported communication options for the target resource.
- TRACE — echoes the received request back to the client.
- CONNECT — establishes a tunnel, typically used with HTTPS proxies.

33.3 HTTP status codes

Range	Meaning
1xx	Informational
2xx	Success
200	OK
201	Created
204	No Content (used for DELETE)
3xx	Redirection (rarely used in APIs)
4xx	Client errors
400	Bad Request
401	Unauthorized
403	Authorized but no rights
404	Not Found
405	Method Not Allowed
406	Not Acceptable / wrong format
409	Conflict / concurrency issue
415	Unsupported Media Type
422	Unprocessable Entity / validation error
5xx	Server errors
500	Internal Server Error

JSON is the de facto response format but is not part of the REST specification — XML is equally valid. The negotiation of response format is called **content negotiation**.

33.4 Project setup

1. Create an ASP.NET Core Web API project in Visual Studio.
 - Name the project with `.API` suffix; solution name without (to support multiple projects).
 - Remove default boilerplate (WeatherForecast controller).
 - Change debug profile to console app; disable launch browser; set a static URL (e.g., `localhost:51044`).
2. Set up the data store.
 - Add NuGet packages: `EFCore`, `EFCore.SqlServer`, `EFCore.Tools`.
 - Create models/entities.
 - Create `DbContext`.
 - Create repository interface and EF implementation.
 - Register services in `Startup.ConfigureServices`.
 - Run `Add-Migration InitialMigration`.
 - Configure logging in `appsettings.json`:

```
{
  "Logging": {
    "Console": {
      "LogLevel": {
        "Microsoft.Hosting.Lifetime": "Trace"
      }
    }
  }
}
```

```
    }
  },
  "AllowedHosts": "*"
}
```

3. Create controllers.

- Implement `ActionResult` action methods.
- Add routing attributes.
- Configure endpoint routing in `Startup.Configure()`.
- Test with Postman — verify status codes.
- In `ConfigureServices`, accept formats beyond JSON.

4. Use DTOs (Data Transfer Objects).

- Change return types from `ActionResult` to `ActionResult<IEnumerable<AuthorDto>>`.
- Use AutoMapper (`AutoMapper.Extensions.Microsoft.DependencyInjection`) instead of manual mapping:

```
namespace CourseLibrary.API.Profiles
{
    public class AuthorsProfile : Profile
    {
        public AuthorsProfile()
        {
            CreateMap<Entities.Author, Models.AuthorDto>()
                .ForMember(
                    dest => dest.Name,
                    opt => opt.MapFrom(src => $"{src.FirstName} {src.LastName}"))
                .ForMember(
                    dest => dest.Age,
                    opt => opt.MapFrom(src => src.DateOfBirth.GetCurrentAge()));
        }
    }
}
```

5. Error handling.

- Do **not** use exceptions for expected validation failures — they are expensive and can be abused to DoS the API.
- Do **not** expose stack traces in production — they reveal internal implementation and are useless to API consumers.
- Set `ASPNETCORE_ENVIRONMENT` to `Production` in project properties.
- Add a global exception handler in `Startup.Configure()`:

```
app.UseExceptionHandler(appBuilder =>
{
    appBuilder.Run(async context =>
    {
        context.Response.StatusCode = 500;
        await context.Response.WriteAsync("An unexpected error occurred. Please try again.");
    });
});
```

6. Filtering and searching.

- Filtering — use when you know which items are in a collection and want to narrow results by field value.
- Searching — use when you do not know exactly which items exist and want to match by query string.